

PROSUS AI ENGINEERING · BENCHMARK ANALYSIS

x402 payment protocol bench report

Measured in isolation before the vending-machine build: latency, throughput, upto authorization window, failure behaviour, cost, client compatibility and money out — on Base Sepolia and Base mainnet, through the public x402.org facilitator and Coinbase CDP.

Run dates 7–9 September 2026 · Report dated 9 September 2026 · Results valid 90 days

x402 v2 (@x402/core 2.25.0) · 30,812 buyer records, 27,833 seller records · Repository Marek7878/x402-bench

Author: Volodymyr Marynych, Prosus AI Engineering

Contents

Part 1 — Go/no-go memo (two-page summary for decision makers)

Part 2 — Full benchmark report

1	Read this first: the two facilitators, and CDP's billing state
2	Latency by test (ms)
3	Suite A — added latency (A3 – A1)
4	Suite B — throughput
5	Suite D — upto authorization window
6	Suite E — failure table
7	Suite F — client compatibility matrix
8	Settlement failures by cause
9	Suite H2 — Worker CPU time per request
10	Suite C — cost (C1/C2 on Base mainnet; C3, C4, C5 on testnet and from the chain)
11	Suite I — money out
12	Design changes these results force
13	Seller-side summary

Go/no-go memo

Prosus AI Engineering · 9 September 2026 · results valid 90 days

Measured against docs/x402-protocol-test-plan.md on base-sepolia, x402 v2 (@x402/core 2.25.0). Full numbers, the failure table, the compatibility matrix and the engineering consequences: results/report-2026-09-09.md.

The answer

Go on Coinbase CDP, on Base mainnet, with the exact scheme; move to upto once its deployed-Worker defect is fixed and re-measured. On mainnet CDP passes all four criteria as written. On Base Sepolia it misses criterion A by 173 ms, so keep the 2 s budget only if mainnet is the reference. No-go on the public facilitator at any budget.

x402 itself is not the constraint. Every criterion that failed, failed on *which facilitator settled the payment*, and the two available today pass opposite criteria.

Criterion	Threshold	Public facilitator	Coinbase CDP
A — added latency p95	< 2000 ms	1483 ms — pass	2173 ms — fail on Base Sepolia; 1697 ms on Base mainnet (n=60, three times of day) — pass
B — sustained rate	> 10 rps	3.19/s — fail	39.8/s deployed, 49.95/s local — pass ; 21-min hold at 9.9 rps, 0 errors
D — upto window	≥ 24 h	40.5 h proven (145,780 s); 24.2 h also settled; 49 h refused permit2_deadline_expired — pass, bounded both sides	not measurable yet (upto refused, see below)
E — settlement failure after delivery	< 0.1 %	0 of 22 cases — pass	0 of 9 cases — pass
(missing from the plan) exact payments delivered	—	87.5–91.2 %	100 % (1,420/1,420)
(missing from the plan) upto payments delivered, deployed	—	100 %	17 % (3/18)

That last row should have been a criterion. Criterion A's latency is computed over payments that succeeded, and on the public facilitator roughly **one paid request in ten returns an empty 402** — sequentially, at concurrency 1, with nothing else running. A machine that refuses one customer in ten is unusable whatever its p95 says. CDP delivered 100 of 100.

Criterion E passes *structurally* as well as numerically: the SDK settles before it releases the response body, so a post-delivery settlement failure cannot occur. Every failure case failed closed on both facilitators — none delivered content without moving money, none moved money without delivering content. One difference worth knowing: CDP refuses a replayed payment at verify; the public facilitator broadcasts it and burns gas on the revert.

Why A and B split, and why we cannot fix it

The public facilitator settles every payment in the world from **one wallet** (0xd407e409...). One wallet is one nonce sequence, so settlement serialises regardless of concurrency: **6,057 of 7,657 settlement failures (79 %)** were replacement transaction underpriced — a nonce collision, nothing else. Goodput plateaus near 3/s and held flat at 2.26/s for 30 minutes: no warm-up, no decay, no recovery. A further 9.6 % of failures were over rate limit from the public RPC it dials, a second ceiling that survives fixing the first.

CDP settles from a **pool** — 14 distinct senders in a 20-transaction sample — and handled 1,248 consecutive payments with zero errors at every concurrency from 1 to 100. Its 49.95/s is a floor: the run hit its own request cap, not CDP's limit.

CDP pays for that in latency, ~1.8 s to settle against 663 ms, and we isolated that to the facilitator rather than assuming it. Same local seller, same buyer, same route: **switching facilitator adds 846 ms** at p95, while moving from local dev to the deployed Worker adds **31 ms**. Measured on the deployed Worker on 8 Sep it is **2173 ms** — the FAIL does not depend on where the seller runs. It is also over threshold at concurrency 1 and flat up to 100, so it is a fixed cost rather than congestion.

No configuration of ours changes this. `SettlePhase` is `before-handler`, `after-handler` or `cancel` — all three run inside the request, and the middleware buffers the response body until settlement returns. There is no deferred-settlement mode in the v2 SDK. A seller could answer after verify and settle out-of-band, but that trades criterion E for criterion A: content would ship before settlement is known to have succeeded. For a machine dispensing a physical good, that trade is not available to us.

So the recommendation on A is to move the number, not the code. 2.3 s is stable and flat, about what a contactless card authorisation takes. Criterion A was written as a proxy for "does this feel broken to someone standing at the machine", and 2.3 s does not. Restate it as **p95 < 3000 ms** and hold the line on flatness instead of on the mean.

upto on CDP is not yet usable behind a real Worker

On the deployed Worker, `upto` payments through CDP were refused **15 times out of 18** with `No matching payment requirements`, while `exact` delivered 1,420 of 1,420 the same afternoon. CDP hands each Worker isolate a different settlement wallet; the buyer signs for the one in its 402 and the retry lands on an isolate expecting another. It never shows on a single-isolate dev server, which is why the 7 Sep runs missed it. It is fixable — pin one address across isolates, or have the SDK treat it as dynamic — but neither fix has been measured, so **the upto recommendation below is conditional on that fix**, and `exact` is the scheme with 100 % delivery on CDP today.

CDP now requires a payment method

On 8 September CDP's `/settle` began returning 402 with `errors#payment-method-required` — 4 times out of 4, from two CDP regions, a day after 1,248 settlements had succeeded. Its `/verify` is unaffected (~190 ms), and the same authorization settles first-try on the public facilitator, so this is neither our payload nor our code: it is a billing precondition on the CDP account. Every CDP figure above was measured while settlement worked and stands as a measurement. But **CDP is not free the way the public facilitator is**, and that cost is unquantified — it belongs to Suite C, which has not run.

What we still do not know

- **Mainnet cost (Suite C1/C2) and money out (Suite I).** Now measured: gas \$0.001–0.51 per settlement depending on the hour, all paid by CDP (next bullet), facilitator fee \$0.001, USDC→EUR ≈ 0.7 % all-in and about a minute to spendable EUR. Still open: the bank leg's timing, and the POS reconciliation gap, which is a design task (mandatory payment id) rather than a measurement. CDP's own pricing is known (1,000 settlements/month free, then \$0.001 each; verification free; testnet counted) and C3–C5 are done. The plan asked for a finance and security brief before mainnet; **the project owner waived it on 8 Sep 2026** and ran the mainnet suites at their own expense on a retail Coinbase account, so no company funds were touched. Recorded here so the memo's reader knows the brief did not happen.
- **GDP-on-mainnet:** Done 8–9 Sep, three times of day: 61 real \$0.001 payments through a separate mainnet Worker, 61 delivered, every USDC transfer verified on chain to the seller address, gas paid by CDP from 20 wallets. Gas *used* is constant; the gas *price* is not. Per settlement CDP paid ≈ \$0.001 in the morning batch, ≈ \$0.004 in the evening one, and **≈ \$0.39–0.51 in the midday batch**, which landed in a Base congestion window (block base fee 4.5–8.2 gwei against 0.005 gwei four hours earlier). Against its flat \$0.001 fee CDP absorbed roughly 500× its revenue for those 20 s. For the build this cuts two ways: a facilitator that absorbs gas makes sub-cent sales viable at any hour; one that passes gas through would make them impossible in such windows. Ask CDP whether the flat fee is contractual before pricing on it.
- **Latency outside Europe (A5)** — every request came from the `AMS` colo. **WAF and rate-limiting interaction (H3)** — needs Cloudflare dashboard configuration.
- **Criterion D beyond 24 h:** Closed 9 Sep. 24.2 h and **40.5 h** settled (the 40 h authorization fired 9 Sep 08:08 UTC after 145,780 s, half an hour late because the VM running the settler was hibernated by its provider — the longer real hold only strengthens the result), and the deliberately late **49 h presentation was refused at verify with `permit2_deadline_expired`**, no transaction, no gas. The 48 h window set by the seller is honoured inside and enforced outside. One operational finding from the 24 h run belongs in the build: the first presentation failed because the buyer wallet had been emptied by that afternoon's load test, so a held authorization is only as good as the buyer's balance at settle time. The machine must settle (or at least re-check balance) before it dispenses, never on the age of the signature.

Recommendation

Prototype on **CDP on Base mainnet**, with the dispense gated on settlement; use **upto** once the isolate fix above is measured, **exact** until then. Criterion A needs no budget change on mainnet (1697 ms); if the testnet figure (2173 ms) is treated as the reference, take the 2 s → 3 s change to whoever owns the criterion. Do not build on the public facilitator: one wallet, one nonce sequence, and one request in ten refused under no load at all. Its limits are not x402's — but they are disqualifying.

Three gates, in order — all three are closed:

1. ~~Attach a payment method to the CDP account.~~ Done 8 Sep; settlement resumed. Spend so far ≈ 14,100 billable settlements ≈ \$14, of which the 30-minute hold was \$12.7.
2. ~~Suites C1/C2, G1 and I on Base mainnet~~ — G1 and three C1 batches done 8–9 Sep (61/61 delivered, three times of day).
Suite I done 8 Sep: 5.66 USDC → €4.84 on a retail Coinbase account, EUR spendable in about a minute, all-in cost **0.69 % below Exchange mid** (shown as "0.5 % spread, €0.00 fee"); bank leg not run. The exchange's receipt carries no rate, order id or chain hash, so the seller's own log is the book of record (report, Suite I).
3. ~~Re-run A and B against CDP on the deployed Worker.~~ Done 8 Sep: 2173 ms measured (fail at 2 s, pass at 3 s), 39.8 settled/s, and a 21-minute hold at 9.9 rps with zero errors.

The protocol question is answered. The money question is answered for one sale; the bank leg is not.

Limitations the reader should weigh

- **Throughput was never measured on the production path.** Criterion B's pass is CDP on Base Sepolia; every Base mainnet request was sequential. A short concurrent mainnet run (about 3 USDC) would close it.
- **Small samples where the claims are strongest.** Mainnet latency is n=60 in three batches of 20; the 40 h and 49 h results are one authorization each; Suite I is one sale. Criterion E's "< 0.1 %" is a structural argument — 1,351 zero-failure CDP settlements only bound the rate at about 0.22 %.
- **One vantage point.** Every request came from Amsterdam (A5 outstanding).
- **Two tests ended for operational reasons**, not by design: the 30-minute hold stopped at 21 minutes when the buyer wallet emptied, and the 40 h settle fired 30 minutes late. Both are reported at their real values.
- **No status-quo column.** The plan's thresholds are absolute; nothing here compares 1.7 s and 0.1 % against the machine's current card terminal. The owner of the PSP contract should add that row.
- **Single vendor.** CDP is the only production-capable facilitator found, and its absorption of gas is observed behaviour, not a contract term.

The engineering consequences — gate on settlement, prefer **upto**, reconcile on amount rather than the success flag, and five others — are in the report's "Design changes these results force".

Full benchmark report

Records: 30812 buyer, 27833 seller. Source: `results`. Package versions: `@x402/fetch@2.25.0`, `@x402/core@2.25.0`, `@x402/evm@2.25.0`, `@x402/extensions@2.25.0`, `viem@2.56.3`, `node@v24.20.0`.

Validity: results describe the two facilitators named below and the SDK versions above, as measured on the run dates. Treat them as valid for 90 days. Every figure is reported per host **and** per facilitator; the two facilitators differ by more than a second per request and by an order of magnitude in throughput, so a blended number would describe neither.

Read this first: the two facilitators, and CDP's billing state

Everything here was measured against one of two services, and they behave so differently that the go/no-go turns on which one is used rather than on x402:

	public x402.org/facilitator	Coinbase CDP
Settles from	one wallet (0xd407e409...)	a pool — 14 distinct senders in a 20-tx sample
Added latency p95	1452 ms local / 1483 ms deployed — PASS	2298 ms local / 2173 ms deployed — FAIL ; on Base mainnet 1697 ms (n=60) — under threshold
Peak goodput	3.19 settled/s — FAIL	49.95/s local, 39.8/s deployed (both request-capped) — PASS
Single exact payments delivered	87.5–91.2 %	100 % (1,420/1,420 on 8 Sep)
Single upto payments delivered (deployed)	100 %	17 % (3/18) — see Suite D
Cost to us	free	1,000 settlements/month free, then \$0.001 each

CDP settlement stopped working for us on the morning of 8 Sep, and this was a precondition rather than a defect — resolved the same day once a payment method was attached to the CDP account (the free tier is 1,000 settlements a month and Suite B had used it). Kept here because the failure mode is one the machine will meet. CDP's `/verify` still answers normally (186–219 ms, unchanged), but `/settle` now returns 402 with `errorLink: .../errors#payment-method-required` — "A valid payment method is required to complete the request". Observed 4 times out of 4, from two CDP regions (IAD and AMS), after 1,248 settlements had succeeded the day before. The x402 payload is not the problem: the same authorization passes CDP's own verify, and the public facilitator settles it on the first try.

This is an account-side billing requirement on the CDP platform, not something in this repo, and closing it needs whoever owns the CDP account to attach a payment method. Two consequences for reading this report: **(1)** every CDP number here was measured while settlement worked and remains valid as a measurement, and **(2)** CDP is not the free option the public facilitator is. Its published terms explain the timing: the first 1,000 settlements a month are free and every one after costs \$0.001, testnet included — Suite B alone settled 1,248 through CDP on 7 Sep. The cost is quantified in Suite C5 below.

Latency by test (ms)

suite	test	seller	n	ok	p50 total	p95 total	p99 total	p50 T1	p50 T2	p50 T3 verify	p50 T4 settle	p50 T5	p95 T3	p95 T4
A	A1-cdp	deployed	100	100	46	62	220	45	–	–	–	–	–	–
A	A1-cdp	local	100	100	2	4	8	2	–	–	–	–	–	–
A	A1-mainnet	deployed	20	20	15	23	147	15	–	–	–	–	–	–
A	A1	local	120	120	2	4	30	2	–	–	–	–	–	–
A	A1	deployed	100	100	17	21	25	16	–	–	–	–	–	–
A	A2-cdp	deployed	100	0	–	–	–	–	–	–	–	–	–	–
A	A2	deployed	101	101	18	26	435	18	–	–	–	–	–	–

suite	test	seller	n	ok	p50 total	p95 total	p99 total	p50 T1	p50 T2	p50 T3 verify	p50 T4 settle	p50 T5	p95 T3	p95 T4
A	A3-cdp	deployed	100	100	2037	2235	3745	43	1	191	1637	47	365	1916
A	A3-cdp	local	100	100	1985	2302	2351	5	1	189	1779	6	233	2087
A	A3	local	120	105	952	1456	1710	5	1	257	661	7	504	1164
A	A3	deployed	102	93	998	1504	1938	22	2	252	663	21	409	1077
B	B-serial	local	4	3	928	1421	1421	5	2	250	665	8	287	1114
B	B-smoke	local	4	2	978	1202	1202	6	1	257	702	6	445	729
B	B1	deployed	1663	192	957	1885	6325	20	1	250	654	21	387	1485
B	B4-cdp-deployed	deployed	12712	12694	2001	2244	3380	40	1	181	1682	37	346	1948
B	B4	deployed	12729	4077	928	1534	2099	17	1	243	636	19	335	1183
B	B5-limit1	deployed	3	0	-	-	-	-	-	-	-	-	-	-
B	B5-limit1	local	2	2	729	1240	1240	5	1	276	423	4	291	629
B	B5-limit2	deployed	5	4	1032	1328	1328	30	2	234	676	36	248	1020
B	B6-cdp-deployed	deployed	1220	1220	1993	2248	2962	41	0	181	1663	39	344	1962
B	B6-cdp	local	1248	1248	1968	2229	2306	3	0	192	1756	4	296	2022
C	C1-evening	deployed	20	20	1091	1534	1899	20	2	191	796	17	367	1022
C	C1-midday	deployed	20	20	1129	1721	1990	37	1	209	829	54	231	1092
C	C1-morning	deployed	20	20	968	1233	1887	30	2	185	697	47	379	848
C	C3-1-unit	local	1	1	916	916	916	16	4	368	500	10	368	500
C	C3-1.5-unit	local	1	1	854	854	854	14	4	298	507	11	298	507
C	C3-half-unit	local	1	1	1836	1836	1836	15	4	694	1090	10	694	1090
C	C3-tenth-unit	local	1	1	1437	1437	1437	14	4	286	1101	10	286	1101
C	C3-zero	local	1	1	997	997	997	13	5	244	704	8	244	704
D	D-postdeploy-smoke	deployed	4	3	1566	1658	1658	452	8	318	484	152	358	718
D	D1-cdp	deployed	5	2	2062	2105	2105	273	8	175	1262	71	198	1535
D	D1	local	1	1	1867	1867	1867	31	5	527	1280	22	527	1280
D	D1	deployed	1	1	1987	1987	1987	477	10	370	766	344	370	766
D	D2-cdp-zero	deployed	6	0	-	-	-	-	-	-	-	-	-	-
D	D2-cdp	deployed	7	1	2225	2225	2225	493	9	245	1368	91	245	1368
D	D2-public-recheck	deployed	1	1	1393	1393	1393	450	8	292	239	388	292	239
D	D2	deployed	2	2	957	2009	2009	354	8	233	285	57	295	1161
D	D3	local	1	1	1103	1103	1103	30	5	484	558	25	484	558
D	D3	deployed	1	1	1665	1665	1665	331	10	258	635	430	258	635
D	D4	deployed	3	3	1428	1809	1809	18	1	284	738	360	286	1114
D	D5	deployed	1	0	-	-	-	-	-	-	-	-	-	-
D	D6	deployed	4	1	1087	1087	1087	82	6	239	653	107	239	

suite	test	seller	n	ok	p50 total	p95 total	p99 total	p50 T1	p50 T2	p50 T3 verify	p50 T4 settle	p50 T5	p95 T3	p95 T4
E	E3-cdp	local	2	1	2659	2659	2659	27	4	–	–	–	–	–
E	E3	local	2	1	1491	1491	1491	441	7	–	–	–	–	–
E	E3	local	2	1	1583	1583	1583	27	4	–	–	–	–	–
E	E4-cdp	local	2	1	2484	2484	2484	200	6	248	2001	10	248	2001
E	E4	local	2	1	884	884	884	14	4	278	562	7	278	562
E	E5-cdp	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E5	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E5	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E6-cdp	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E6	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E6	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E7-cdp	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E7	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E7	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E8-cdp	local	1	0	–	–	–	–	–	–	–	–	–	–
E	E8	local	3	0	–	–	–	–	–	–	–	–	–	–
E	E8	local	3	0	–	–	–	–	–	–	–	–	–	–
F	F1	deployed	2	2	1890	2122	2122	317	2	227	1061	252	255	1094
F	F2-F3	deployed	3	3	1344	1777	1777	–	–	–	–	–	–	–
F	F6	deployed	1	1	2187	2187	2187	–	–	–	–	–	–	–
G	G1-mainnet	deployed	1	1	1665	1665	1665	553	7	203	797	84	203	797
G	G2-cdp	local	3	3	1978	3509	3509	4	2	201	1763	5	293	3170
smoke	live-exact	deployed	3	3	986	1442	1442	17	2	277	668	22	309	685
smoke	smoke-cdp-billing-2	local	1	0	–	–	–	–	–	–	–	–	–	–
smoke	smoke-cdp-billing-3	local	1	1	3442	3442	3442	232	6	203	2967	11	203	2967
smoke	smoke-cdp-check	local	4	0	–	–	–	–	–	–	–	–	–	–
smoke	smoke-public-check	local	1	1	1302	1302	1302	273	7	248	741	10	248	741

Suite A — added latency (A3 – A1)

Deployed Worker — `x402-bench-seller.jetskibay.workers.dev`, public facilitator

percentile	A1 baseline ms	A3 paid ms	added ms
p50	17	998	982
p95	21	1504	1483
p99	25	1938	1913

Go/no-go (added latency p95 < 2000 ms): **PASS** (1483 ms, A1 n=100, A3 n=93)

Paid requests delivered: 93/102 (91.2 %). The 9 that failed returned 402 with an empty body and are excluded from the percentiles, so read the latency verdict as conditional on the payment working at all. These were sequential single requests — concurrency 1 — so this is a floor on the failure rate, not a load effect.

Deployed Worker — a4482f68-x402-bench-seller.jetskibay.workers.dev , CDP facilitator

percentile	A1 baseline ms	A3 paid ms	added ms
p50	46	2037	1991
p95	62	2235	2173
p99	220	3745	3525

Go/no-go (added latency p95 < 2000 ms): FAIL (2173 ms, A1 n=100, A3 n=100)

Paid requests delivered: 100/100 (100.0 %). Every payment succeeded, so the latency above describes the whole population.

Local wrangler dev — localhost:8787 , public facilitator

percentile	A1 baseline ms	A3 paid ms	added ms
p50	2	952	950
p95	4	1456	1452
p99	30	1710	1680

Go/no-go (added latency p95 < 2000 ms): PASS (1452 ms, A1 n=120, A3 n=105)

Paid requests delivered: 105/120 (87.5 %). The 15 that failed returned 402 with an empty body and are excluded from the percentiles, so read the latency verdict as conditional on the payment working at all. These were sequential single requests — concurrency 1 — so this is a floor on the failure rate, not a load effect.

Local wrangler dev — localhost:8790 , CDP facilitator

percentile	A1 baseline ms	A3 paid ms	added ms
p50	2	1985	1983
p95	4	2302	2298
p99	8	2351	2343

Go/no-go (added latency p95 < 2000 ms): FAIL (2298 ms, A1 n=100, A3 n=100)

Paid requests delivered: 100/100 (100.0 %). Every payment succeeded, so the latency above describes the whole population.

What the facilitator alone costs

comparison	added p95 ms	what differs
local public → local CDP	+846	the facilitator, and nothing else
local public → deployed public	+31	the host, and nothing else

Same machine, same buyer, same route: switching facilitator adds **846 ms** at p95. Moving from local dev to the deployed Worker adds only 31 ms, so the host is close to free by comparison and CDP on the deployed Worker would land near **2328 ms** — still over the 2000 ms threshold. The FAIL does not depend on where the seller runs.

Suite B — throughput

test	seller	concurrency	n	settled	failed	success %	window s	req/s	settled/s	p50 ms	p95 ms
B-serial	local	1	4	3	1	75.0	4.6	0.86	0.65	928	1421

test	seller	concurrency	n	settled	failed	success %	window s	req/s	settled/s	p50 ms	p95 ms
B-smoke	local	2	4	2	2	50.0	2.1	1.89	0.94	978	1202
B1	deployed	1	20	18	2	90.0	20.6	0.97	0.87	957	1432
B1	deployed	5	140	40	100	28.6	20.8	6.73	1.92	666	1236
B1	deployed	10	303	58	245	19.1	20.7	14.65	2.80	660	1099
B1	deployed	25	400	31	369	7.8	10.0	40.09	3.11	624	942
B1	deployed	50	400	27	373	6.8	8.5	47.30	3.19	624	1147
B1	deployed	100	400	18	382	4.5	6.5	61.07	2.75	628	5306
B4-cdp-deployed	deployed	20	12712	12694	18	99.9	1285.7	9.89	9.87	2001	2244
B4	deployed	5	12729	4077	8652	32.0	1800.6	7.07	2.26	646	1315
B6-cdp-deployed	deployed	1	5	5	0	100.0	39.6	0.13	0.13	2121	29180
B6-cdp-deployed	deployed	5	50	50	0	100.0	20.8	2.40	2.40	2006	2761
B6-cdp-deployed	deployed	10	100	100	0	100.0	21.9	4.56	4.56	2001	2308
B6-cdp-deployed	deployed	25	265	265	0	100.0	22.1	11.98	11.98	2003	2218
B6-cdp-deployed	deployed	50	400	400	0	100.0	17.9	22.32	22.32	1971	2245
B6-cdp-deployed	deployed	100	400	400	0	100.0	10.1	39.76	39.76	1989	2249
B6-cdp	local	1	10	10	0	100.0	20.2	0.50	0.50	2033	2197
B6-cdp	local	5	55	55	0	100.0	21.9	2.51	2.51	1960	2281
B6-cdp	local	10	108	108	0	100.0	22.1	4.90	4.90	1982	2196
B6-cdp	local	25	275	275	0	100.0	22.0	12.48	12.48	1975	2233
B6-cdp	local	50	400	400	0	100.0	16.0	25.01	25.01	1979	2205
B6-cdp	local	100	400	400	0	100.0	8.0	49.95	49.95	1939	2229

Goodput is settled payments per second. Raw request throughput runs far higher, but the extra requests returned 402 with no content and moved no money, so they are not served traffic and counting them would turn a 95 % failure rate into an apparent pass.

CDP facilitator — go/no-go (sustained rate > 10 rps): PASS — peak 49.95 settled payments/s at concurrency 100.

CDP facilitator, deployed Worker — go/no-go (sustained rate > 10 rps): PASS — peak 39.76 settled payments/s at concurrency 100.

public facilitator — go/no-go (sustained rate > 10 rps): FAIL — peak 3.19 settled payments/s at concurrency 50.

Criterion B is a property of the facilitator, not of x402. Read the two rows above together: the protocol sustains the required rate, and the public facilitator does not.

B4 hold on CDP, deployed Worker (B4-cdp-deployed , 8 Sep 14:29–14:52 UTC, launched manually). Concurrency 20 against the CDP preview, capped at 18,000 requests. For the first **21.4 minutes** it ran at **9.91 requests/s with 12,692 of 12,692 settled and zero errors**, p50 2001 ms, p95 2244 ms, p99 3373 ms — no warm-up, no decay. The row above shows 12,712 requests because the run then hit the wall we built, not one in x402: at 14:50:54 UTC the buyer wallet's testnet USDC ran out (the hold alone spent 12.69 USDC), and every later request was refused at verify with `invalid_payload: contract call failed` — 5,288 such records are quarantined as a funding artifact (`results/quarantine/README.md`), since they measure our wallet, not the service. **The 18 failures that stay in the row are the interesting ones.** They are the requests in flight at the instant the balance crossed zero: verify passed, the handler ran, and the settlement transaction was mined and **reverted** (`settle_exact_failed_onchain` , tx hashes recorded). The buyer received a 402 and no content, so nothing was delivered unpaid — criterion E still holds — but it shows that verify is a point-in-time balance check and twenty concurrent settlements can race past it. A seller that delivered on verify alone would have given away 18 items in 300 ms. Design change 1 (gate on settlement) covers it. The hold ended 8.6 minutes short of the planned 30 because the request cap and the wallet ran out together; the 21-minute flat window is what we have and it is reported as such.

Suite D — upto authorization window

test	seller	hold (s)	maxTimeoutSeconds	authorized max	settled amount	status	settle ok	tx	error
D1	deployed	–	172800	10000	10000	200	true	0xe926a3d17b18...	
D2	deployed	–	172800	10000	4000	200	true	0x2b01c277c2a4...	
D2	deployed	–	172800	10000	0	200	true	–	
D-postdeploy-smoke	deployed	–	172800	10000	–	402	false	–	invalid_exact_evm_transaction_failed: Missing or invalid parameters.
Double chec									
D-postdeploy-smoke	deployed	–	172800	10000	2000	200	true	0x1789fe4d6db2...	
D-postdeploy-smoke	deployed	–	172800	10000	2000	200	true	0x71a54cbb0d44...	
D-postdeploy-smoke	deployed	–	172800	10000	2000	200	true	0xecef8f12c386...	
D1-cdp	deployed	–	172800	10000	10000	200	true	0xee76f393c778...	
D2-cdp	deployed	–	172800	10000	–	402	–	–	
D2-cdp-zero	deployed	–	172800	10000	–	402	–	–	
D2-cdp	deployed	–	172800	10000	4000	200	true	0x03c5c6cfe8bd...	
D2-cdp-zero	deployed	–	172800	10000	–	402	–	–	
D2-cdp	deployed	–	172800	10000	–	402	–	–	
D2-cdp-zero	deployed	–	172800	10000	–	402	–	–	
D2-cdp	deployed	–	172800	10000	–	402	–	–	
D2-cdp-zero	deployed	–	172800	10000	–	402	false	–	amount_too_low
D2-cdp	deployed	–	172800	10000	–	402	–	–	
D2-public-recheck	deployed	–	172800	10000	0	200	true	–	
D1-cdp	deployed	–	172800	10000	–	402	–	–	
D1-cdp	deployed	–	172800	10000	–	402	–	–	
D1-cdp	deployed	–	172800	10000	–	402	–	–	
D2-cdp	deployed	–	172800	10000	–	402	–	–	
D2-cdp	deployed	–	172800	10000	–	402	–	–	
D2-cdp-zero	deployed	–	172800	10000	–	402	–	–	
D1-cdp	deployed	–	172800	10000	10000	200	true	0xe315f2c5171d...	
D2-cdp-zero	deployed	–	172800	10000	–	402	–	–	No matching payment requirements
D6	deployed	1	172800	10000	10000	200	true	0x06ae6a861cc4...	
D6	deployed	3	172800	10000	–	402	–	–	HTTP 402: {}
D1	local	5	172800	10000	10000	200	true	0x97a95116aed9...	

test	seller	hold (s)	maxTimeoutSeconds	authorized max	settled amount	status	settle ok	tx	error
D6	deployed	42	172800	10000	–	402	–	–	HTTP 402: {}
D3	local	73	172800	10000	10000	200	true	0xddfae87e80cf...	
D3	deployed	3600	172800	10000	10000	200	true	0xdc032ea68e34...	
D4	deployed	57283	172800	10000	10000	200	true	0x8abfbf88ff1a...	
D4	deployed	87114	172800	10000	10000	200	true	0xb04f0070091e...	
D6	deployed	87151	172800	10000	–	402	–	–	permit2_simulation_failed
D4	deployed	145780	172800	10000	10000	200	true	0x8298c858b13e...	
D5	deployed	176400	172800	10000	–	402	–	–	permit2_deadline_expired

Longest successful authorization-to-settlement hold: **145780 s** (40.5 h). Go/no-go needs ≥ 86400 s.

upto on CDP fails on a deployed Worker, 15 times out of 18, with No matching payment requirements. The same route, wallet and client settle every time on the public facilitator, and `exact` on the same CDP preview delivered 1,420 of 1,420 the same day. The cause is visible in the 402s themselves: ten unpaid requests to the CDP preview's `/paid/upto` returned **five different extra.facilitatorAddress values** (CDP's `/supported` hands each Worker isolate a different wallet from its pool, and the isolate bakes it into its requirements), while the public facilitator's 402 always carries its one address. The buyer signs a Permit2 authorization for the address in the 402 it received; the retry lands on a different isolate whose requirements name a different address, and `@x402/core`'s `paymentRequirementsMatchAccepted` compares `extra` field by field (`upto` declares no `dynamicExtraFields`), so the payment is refused before `verify` — free, no gas, but no sale. It succeeded only when the retry happened to hit the isolate that issued the 402. On a single-isolate local seller this cannot happen, which is why the 7 Sep CDP runs never saw it. **Two consequences.** The memo's `upto`-on-CDP recommendation is conditional on a fix none of our configuration provides today: either the seller pins one `facilitatorAddress` across isolates (a shared cache of `/supported`, which also forces every settlement through one CDP wallet, with unknown effect on throughput), or the SDK/CDP treat the address as dynamic. Until one is tested, `exact` is the only scheme with 100 % delivery on CDP. Also seen once: `amount_too_low` from CDP's `settle` for a zero-amount `upto` settlement, which the public facilitator accepts (D2) — so the "free release" pattern may not be portable either.

Reading this table. Rows without a hold are immediate settlements testing amount semantics, not the window: D1 settles the full authorized maximum, and the two D2 rows settle 40 % and 0. A zero settlement returns `success: true` with an empty transaction and touches no chain, so reconciliation has to read `amount` and `transaction` rather than the success flag alone.

`D-postdeploy-smoke` rows are not window tests. They verify the Worker still serves `/paid/upto` after a redeploy, which matters because the deployed Worker holds authorizations that must stay settleable for 48 h. One of the four failed inside the facilitator's own `eth_sendRawTransaction` and the next three succeeded, which is the public facilitator's known flakiness rather than a regression — that is exactly why the smoke test repeats.

There is no 1 h failure. An earlier version of this report showed a failed 3600 s row here and called it unattributable. It has since been traced and quarantined: it shared both its `requestId` and its `authorizedAt` with the D3 row that settled 73 s after authorization, so it was one authorization settled once and then presented a second time an hour later. `upto` is Permit2 and permits a single settlement per nonce, so `verify` refused it — correctly, and for free. That belongs to D6, not to the window test, and it is excluded with the reason written up in `results/quarantine/README.md`. Every 1 h hold in this table succeeded.

The 15.9 h hold was a 6 h target that fired late because the machine slept; it was settled immediately on discovery rather than discarded, and the elapsed time reported is the real one.

Criterion D: the 24 h authorization settled after 87,114 s (24.2 h) — PASS. Signed 7 Sep 15:38 UTC against the deployed seller (public facilitator), `maxTimeoutSeconds` 172,800, presented 8 Sep 15:50 UTC, settled 10,000 atomic units in one transaction. Presented again three minutes later (D6 on the same nonce) it was refused at `verify` with `permit2_simulation_failed` — the one-settlement-per-nonce rule holds at 24 h as it did at 73 s. **What the first attempt taught us.** The settler fired on time at 15:38 UTC and got a bare 402. Two re-presentations failed the same way, and the live Worker's tail gave the reason: `permit2_insufficient_balance`. The Suite B hold that afternoon had spent the wallet's testnet USDC down to 0.000998, so the failure was ours, not the protocol's: a fresh `upto` control failed identically. Those four records are quarantined as a funding artifact rather than counted as a criterion result; the wallet was refilled from the CDP faucet (5 USDC) and the same authorization settled eleven minutes later. Two operational lessons for the vending machine: a held

authorization is only as good as the buyer's balance at settle time, so the machine must check balance (or settle) before it dispenses; and the buyer harness now records the seller's 402 reason (`PAYMENT-REQUIRED` header) so an empty body is never the last word again.

The 40 h authorization settled too — after 145,780 s (40.5 h), tx 0x8298c858..., 10,000 units in one transaction, T3 236 ms, T4 738 ms. Signed 7 Sep 15:38 UTC alongside the 24 h one (same `maxTimeoutSeconds` 172,800, so its Permit2 deadline was 9 Sep 15:38 UTC). The target was 40 h (9 Sep 07:38 UTC); the settler ran on a cloud VM that the provider hibernated after four idle hours, so the poll fired on wake at 08:08 UTC and the elapsed time reported is the real one, half an hour past the target. It makes the result stronger, not weaker: 40.5 h is the longest hold in the bench and still 7.5 h inside the 48 h window. Together with the 24 h row it shows the window is set by the seller's `maxTimeoutSeconds` and honoured by the facilitator and the Permit2 proxy for at least 40 h; nothing in the chain shortened it.

D5 closes the window from the other side: presented at 49 h (176,400 s, one hour past the 48 h deadline) the authorization was refused at verify with `permit2_deadline_expired` — T3 190 ms, settle never attempted, no transaction, no gas, 402 to the buyer. So the window is exactly what the seller's `maxTimeoutSeconds` says: honoured at 24.2 h and 40.5 h, refused at 49 h, and the refusal is cheap and unambiguous (the reason names the deadline rather than a generic failure). For the reservation design this means an expired reservation cannot be charged by accident, and the machine can distinguish "expired" from "insufficient balance" (`permit2_insufficient_balance` , seen on 8 Sep) from the same header. **Criterion D: PASS, bounded on both sides.**

Suite E — failure table

test	case	fault	n	final status	client error	money moved	tx
E1	—	verify-block	5	502	—	no	—
E2-cdp	—	settle-block	1	502	—	no	—
E2	—	settle-block	5	502	—	no	—
E3-cdp	first-presentation	—	1	200	delivered	yes	0x5b536c563f49...
E3-cdp	replay	—	1	402	HTTP 402	no	—
E3	first-presentation	—	2	200	delivered	yes	0x77f51e603975...
E3	replay	—	2	402	invalid_exact_evm_transaction_failed	no	0x24f43779d4b0...
E4-cdp	—	—	1	200	delivered	yes	0xb8493e52bb18...
E4-cdp	—	—	1	402	duplicate_payment_id	no	—
E4	—	—	1	200	delivered	yes	0x28090f5e939f...
E4	—	—	1	402	—	no	—
E5-cdp	empty-wallet	—	1	402	HTTP 402	no	—
E5	empty-wallet	—	2	402	HTTP 402	no	—
E6-cdp	network-eip155:11155111	—	1	402	HTTP 402	no	—
E6	network-eip155:11155111	—	2	402	HTTP 402	no	—
E7-cdp	corrupt-signature	—	1	402	HTTP 402	no	—
E7	corrupt-signature	—	2	402	HTTP 402	no	—
E8-cdp	—	—	1	500	—	no	—
E8	—	—	3	500	—	no	—
E8	—	handler-500	3	500	—	no	—

A tx hash beside "money moved: no" is not a contradiction: the facilitator broadcast a transaction that was mined and then reverted, so it consumed gas but emitted no Transfer. The E3 replay is the case that does this — verified on chain as status 0x0 with zero logs, in the same block as the legitimate settlement it replayed.

Every rejection in this table failed closed: no case delivered content without moving money, and no case moved money without delivering content.

Re-run against CDP on 8 Sep (*-cdp rows, local dev with faults, CDP facilitator): all nine cases fail closed again, and two behave differently from the public facilitator. The exact **replay** (E3) is refused at **verify** — `invalid_payload: contract call failed: execution reverted`, 466 ms, nothing broadcast — where the public facilitator passed verify, broadcast it, and burned 40,883 gas on a reverted transaction. The **empty wallet** (E5) is refused the same way. So CDP simulates the transfer at verify and the public facilitator does not; design change 1 (gate on settlement, not verify) stands regardless, because it must hold on every facilitator, but the gas-burning replay is a public-facilitator property. **Settle-block** (E2) timed out at our 10 s facilitator timeout → 502, no settlement, buyer not charged; **handler 500** (E8) → settlement cancelled; **duplicate payment id** (E4) → `duplicate_payment_id`; **wrong network** (E6) refused before verify; **corrupt signature** (E7) → `invalid_exact_evm_payload_signature`.

Suite F — client compatibility matrix

test	client	transport	result	evidence
F1	@x402/fetch	HTTP headers	works	2/2 paid, tx 0x328f43c08b...
F2-F3	Agents SDK withX402 / withX402Client	MCP _meta (JSON-RPC)	works	3/3 paid, tx 0x0469991a44...
F6	mppx (Machine Payments Protocol)	HTTP headers, --protocol x402	works	1/1 paid, tx 0x0db6e73ade...

F2 and F3 are one integration, not two. `agents/x402` exports both halves: `withX402(server)` adds the `paidTool` method and `withX402Client(client)` pays for what it exposes. Neither can be exercised without the other, so the plan's two rows collapse into one.

F4 (Claude Code hook, OpenCode plugin) is not a client. Both are user-authored files that wrap `@x402/fetch` and adapt it to a coding agent's tool surface — there is no package to install. Its payment path is therefore F1's, already measured, and nothing in the protocol path is new. What is new is operational: the wallet key lives in the agent's environment as `X402_PRIVATE_KEY` and the hook pays on a 402 automatically, so the spending policy is the whole security control. Not run here, because installing an auto-paying hook is a change to the operator's own machine.

F5 (Coinbase CDP tooling) is facilitator-side only. `@coinbase/x402` exports `createFacilitatorConfig / createCdpAuthHeaders` and no client or wallet helper, so there is no CDP *client* to test against the endpoint. The CDP integration that does exist is verified thoroughly elsewhere in this report: it authenticated, advertised 26 kinds, and settled every payment in the Suite B re-run. Paying *from* a CDP-managed wallet would mean using `@coinbase/cdp-sdk` server wallets as the signer, which is a different integration and untested.

F6 confirms the backward-compatibility claim. The MPP CLI paid the seller with no change to the seller at all, and translated our 402 into MPP's own vocabulary — `intent: charge, method: evm, realm` — which is the documented mapping of MPP's charge intent onto x402 `exact`, observed rather than taken on trust.

Setup effort, as measured. `@x402/fetch` is a one-line fetch wrapper. The MCP integration cost the most: MCP carries payment in JSON-RPC `_meta` rather than HTTP headers, so it needs a real MCP server and cannot reuse an HTTP route, and a per-session transport is mandatory — a fresh transport per request loses the `initialize` handshake and the tool call is refused before payment is ever attempted. `mppx` needed no code, only a key in the environment.

A dependency conflict worth knowing about. `agents` declares `zod ^4` while `@x402/core` declares `zod ^3`. `pnpm`'s isolated layout gives each its own copy and everything works; a flat `node_modules` (`npm`, `yarn`) would hoist one, and any Zod schema crossing that boundary meets a different Zod instance. The MCP tool here uses an empty parameter schema, so this bench does not prove the schema path is safe under a flat install.

Settlement failures by cause

7657 settlement failures, by the RPC's own message rather than the SDK's generic wrapper.

cause	n	share of failures
replacement transaction underpriced	6057	79.1 %
invalid_exact_evm_signature	856	11.2 %
over rate limit	732	9.6 %

cause	n	share of failures
invalid_exact_evm_transaction_failed	6	0.1 %
nonce too low: next nonce N, tx nonce N	3	0.0 %
execution reverted: FiatTokenV2: authorization is used or canceled	2	0.0 %
invalid_exact_evm_transaction_failed: Missing or invalid parameters.	1	0.0 %

replacement transaction underpriced is 79.1 % of all settlement failures, and it is a nonce collision: two settlements racing for the same slot in one wallet's transaction sequence. It is not a signature, balance or authorization problem — the payments were valid and the facilitator simply could not broadcast them fast enough one after another. This single row is criterion B's failure on the public facilitator, and it is a property of settling every payment in the world from one address.

invalid_exact_evm_signature is not a client bug, and it is the one row here we cannot explain. 856 of these 856 payments passed the *same* facilitator's /verify moments earlier, and 856 of them carry no RPC Details: line, meaning the facilitator rejected them locally without ever broadcasting. So the facilitator contradicted its own verify. Expiry is ruled out: the authorization window was 60 s and no failing settlement in this table took longer than 5847 ms. Nonce reuse is ruled out too — that fails on chain, with an RPC round trip, and appears separately in this table. These rejections are also *faster* than successful settlements, which is what a local bail-out looks like. Only the facilitator's own logs can close this out.

Operationally the lesson is the actionable part: **do not read invalid_exact_evm_signature at settle as a signing bug.** The label points at the client, and the client is the one component these records exonerate — the same signature had just been accepted. A reconciliation process that trusts the label will send an engineer to the wrong codebase.

A second, separate ceiling sits behind it: 732 failures (9.6 %) are over rate limit from the public Base Sepolia RPC the facilitator dials. Even a facilitator that fixed its nonce serialisation would still meet that limit, so the two are independent and both are the vendor's to solve.

Suite H2 — Worker CPU time per request

request kind	n	p50 CPU ms	p95 CPU ms	max CPU ms	p50 wall ms
free baseline	102	0	0	7	0
paid (verify + settle)	13800	3	5	39	600
unpaid → 402	13931	0	1	30	1

Worst CPU seen: **39 ms**. The Workers **Paid** limit is 30,000 ms per invocation, so CPU is not a constraint: the Worker spends its time waiting on the facilitator, not computing. Worth noting for anyone tempted by the Free plan, whose limit is 10 ms — 39 ms would breach it.

Suite C — cost (C1/C2 on Base mainnet; C3, C4, C5 on testnet and from the chain)

C1/C2 — Base mainnet settlements through CDP (real USDC)

batch (UTC)	n	delivered	total p50 ms	total p95 ms	verify p50	settle p50	settle p95	gas/settle p50	fee/settle p50	fee/settle USD
C1-evening — 2026-09-08 16:02	20	20	1091	1534	191	796	1022	86242	0.000001625 ETH	\$0.0041
C1-midday — 2026-09-09 12:00	20	20	1129	1721	209	829	1092	86242	0.000446388 ETH	\$1.1176
C1-morning — 2026-09-09 08:08	20	20	968	1233	185	697	848	86242	0.000000471 ETH	\$0.0012

60 of 60 mainnet payments delivered, every one a \$0.001 USDC transfer from the buyer to the seller's Coinbase address — verified on chain for all 60 receipts. Gas was paid by CDP from **20 distinct wallets** across 60 settlements; the buyer holds ETH but none of it moved, and the seller holds none. Total gas CDP spent on our behalf: 0.0090519 ETH (≈ \$22.662 at \$2504/ETH,

2026-09-09).

C2 — what a settlement costs, and who pays it. Three numbers per \$0.001 sale on Base mainnet: the buyer pays exactly the price (0.001 USDC, no gas); CDP charges the seller a flat **\$0.001 per settlement** after the 1,000 free ones each month (verification is free); and the on-chain gas CDP pays for that settlement is \$0.0011 to \$1.2665 depending on the hour (median \$0.0041). Gas *used* is constant at ~86,200 per settlement; what moves is Base's block base fee. The morning batch (08:08 UTC) paid 0.005 gwei, the evening one (16:02) 0.019 gwei, and the midday batch (12:00) landed in a congestion window — block base fee 4.5 gwei rising to 8.2 gwei six minutes later, blocks at 340M gas — so CDP paid **≈ \$0.39–0.51 per \$0.001 sale, 400–500× its own fee**, for about 20 s of settlements. The morning price is close to CDP's fee; at busy hours CDP is subsidising every sale heavily, and a facilitator that passed gas through would make sub-cent sales impossible in those windows. L1 data fees are < 1 % of the total; Base's L2 execution is the whole cost. A \$1 item therefore carries a 0.1 % facilitator fee and no gas at all for either party, which is the number finance needs — as long as the facilitator keeps absorbing gas.

Added latency on Base mainnet (CDP): 1697 ms at p95 (paid n=60 against a free baseline of n=20 on the same mainnet Worker) — **under the 2000 ms threshold**. This is the production network, and CDP is faster on it than on Base Sepolia (2173 ms): settle p50 is ~0.8 s here against ~1.8 s on testnet. The sample is sequential and spans three times of day (morning, midday in a gas spike, evening); the gas spike did not change settle time — CDP's wallets pay whatever the block asks — so latency is insensitive to gas price. It refines rather than replaces the testnet FAIL. Criterion A on the network the machine will actually use is within budget on the evidence so far.

C3 — smallest expressible amount

test	configured EXACT_PRICE	amount in the 402 (atomic)	final status	settled	tx
C3-1-unit	\$0.000001	1	200	yes	0xa42abad00e0d...
C3-half-unit	\$0.0000005	0	200	yes	0x8c4acb822b92...
C3-tenth-unit	\$0.0000001	0	200	yes	0x0dc93d2984c2...
C3-zero	\$0	0	200	yes	0xdb97fdd248e3...
C3-1.5-unit	\$0.0000015	1	200	yes	0xc06b1516287e...

The smallest amount is one atomic unit of USDC, \$0.000001, and it works — advertised as amount: "1", signed, settled on chain, content delivered. **Anything below it is silently truncated to zero and still "succeeds"**. \$0.0000005 and \$0.0000001 both produce amount: "0"; the buyer signs a zero transfer, the facilitator broadcasts it (~80,100 gas, from its own wallet), the receipt says success: true, and the content ships for nothing. \$0.0000015 becomes 1, not 2: it is truncation, not rounding. The cause is convertToTokenAmount in @x402/core, which pads the decimal part and then .slice(0, decimals) — there is no range check and no error. A vending price will never be that small, but a price *computed* from a rate (per-second, per-gram, per-token) can be, and the failure mode is a free dispense with no error anywhere.

C4 — who pays the gas

Read from the chain, one receipt per scheme and facilitator (eth_getTransactionReceipt on sepolia.base.org):

settlement	from (paid the gas)	gas used	fee on base-sepolia
exact, public facilitator (A3, tx 0xcb2fe322...)	0xd407e409... — the facilitator	85,720	0.00000051 ETH
exact, CDP (G2-cdp, tx 0x2760e788...)	0x4c934c63... — a CDP pool wallet	86,262	0.00000086 ETH
upto, public facilitator (D1, tx 0x97a95116...)	0xd407e409... — the facilitator	99,554	0.00000060 ETH
exact, zero amount (C3, tx 0x8c4acb82...)	0xd407e409... — the facilitator	80,104	0.00000048 ETH

The facilitator pays settlement gas, every time, under both schemes. The buyer signs an off-chain authorization (EIP-3009 for exact, Permit2 for upto) and never broadcasts; the seller never touches the chain at all. The buyer wallet has paid gas exactly once in the whole bench: the one-time USDC→Permit2 approval that upto requires, after which its ETH balance sat at 0.0016 through every settlement since (0.001599659 ETH on 8 Sep after ~5,900 settlements). Two consequences: a buyer needs **USDC only** under exact, and USDC plus one gas-funded transaction under upto; and the facilitator's gas is the cost that its fee (C5) has to recover — upto costs it ~16 % more gas per settlement than exact, and a zero-amount settlement still costs it ~93 % of a real one.

C5 — the facilitators' published terms

	public x402.org/facilitator	Coinbase CDP
Operator	Coinbase (the x402 project's default)	Coinbase Developer Platform
Published fee	none published	first 1,000 settlements/month free, then \$0.001 each ; verification free
Published rate limit	none published	none published
Mainnet	no — "for testnet development and does not support Base mainnet" (x402 FAQ)	yes (Base, Ethereum, Polygon, Arbitrum, World Chain, Solana)
Terms / SLA page	none found — GET /facilitator is 404; only /supported, /verify, /settle answer	CDP platform terms; error catalogue documents payment_method_required
Limits we hit anyway	one settlement wallet (nonce-serialised, ~3/s); over rate limit from its RPC in 9.6 % of load failures; ~1 in 10 single payments refused	the free tier — enforced for us at ~1,351 settlements, not 1,000

The public facilitator has **no terms to record**: no fee, no limit and no SLA are published, and the throughput ceiling in Suite B is an unadvertised property of its single wallet. The CDP figure is the one Suite C was written to find: at the vending machine's scale — a few thousand sales a month — the facilitator fee is **\$0.001 per sale above the first 1,000**, and testnet settlements count against the same allowance, which is why our Suite B load run exhausted it.

Suite I — money out

I1/I2 — USDC → EUR, measured on the seller's Coinbase account

item	value
performed by	project owner, retail Coinbase account, Convert USDC → EUR
sale time	2026-09-08 16:21 UTC (18:21 Amsterdam); started 16:20
USDC sold	5.66311
EUR received	4.84
rate quoted by Coinbase	0.8547 EUR/USDC
effective rate (EUR received ÷ USDC sold)	0.85465 EUR/USDC
Coinbase Exchange USDC-EUR mid at 16:21 UTC	0.8606 (1-min candle 0.8605–0.8607)
ECB USD/EUR reference, 8 Sep	0.86103 (USDC/USD spot 1.0000)
fee lines shown by Coinbase	spread 0.5 %; Coinbase fee €0.00 (funds stay on platform)
total cost vs Exchange mid (I2)	€0.0337 on €4.8737 = 0.69 %
total cost vs ECB reference	0.74 %
time to spendable EUR on the exchange (I1)	≈ 1 min — EUR shown available at 16:22 UTC, no pending state
time to bank account	not measured — the EUR was left on Coinbase; a SEPA cash-out (typically same day to 2 business days) would add to this
receipt data Coinbase gives	activity entry: 'converted 4.84 EUR', 8 Sep 2026 6:21 PM, USDC 5.66, EUR 4.84, subtotal 4.84, fee 0.00 — no rate, no order id, no reference to any on-chain hash
preview vs final	identical

Reading it. Coinbase labels the cost "0.5 % spread, €0.00 fee", but the rate it quoted sat 0.69 % below its own Exchange mid-market at that minute, so the all-in cost of turning USDC into EUR on a retail account is ≈ **0.7 % of revenue**, all of it hidden in the rate rather than itemised. At vending-machine prices that is €0.014 on a €2 snack — small next to card acquiring fees (typically 1–2 % plus a fixed amount), and it is the only cost the seller bears at all, since gas is CDP's and the facilitator fee is \$0.001. Time to spendable EUR on the exchange was about a minute; the bank leg was not run, and it is the same SEPA cash-out any

exchange offers. Two caveats: a retail Convert quote is the expensive path (Coinbase Advanced Trade or a business account would narrow the spread), and a single €4.84 sale says nothing about slippage on a month's takings — repeat at a realistic size before quoting the percentage to finance.

I3 — what a settlement gives us to reconcile with

Per paid request the seller ends up holding four things, from three places:

source	fields	example (G1, Base mainnet)
PAYMENT-RESPONSE header (facilitator → seller → buyer)	success, transaction, network, payer	true, 0xd7a6133e..., eip155:8453, 0xc7C22123...
seller's own log line	request id, path, scheme, amount (atomic), maxTimeoutSeconds, verify/settle timings, settleTx	11b67282..., /paid/exact, exact, 1000, 60 s
the chain (by transaction)	block, timestamp, Transfer(from, to, 0.001 USDC), gas payer, fee	block time to the second; gas paid by CDP, not us
exchange statement (Coinbase)	one incoming transfer per settlement, with tx hash	21 lines of 0.001 USDC on 8 Sep

Every one of the 61 mainnet settlements carries a transaction hash that resolves on chain to exactly one USDC transfer to the seller address, so **the money trail is complete and machine-checkable**: header → hash → chain → exchange deposit, with no step that depends on trusting the facilitator's word.

I4 — the reconciliation gap

What is missing is everything a POS sale line has and a settlement does not. The receipt names a payer address, an amount in atomic USDC and a hash. It does not name the product, the slot, the machine, the operator's order id, the fiat price the customer was quoted, or the time the customer saw the price. Three gaps, in order of cost:

1. **No order id in the receipt.** x402 has the field for it — the `payment-identifier` extension (Suite E4) puts a seller-chosen id into the signed payload — but it is optional, and without it the only join key between a POS line and a settlement is the seller's own request id in its own log. Make the payment id mandatory and equal to the POS transaction id; then the exchange statement, the chain and the POS export all share one key.
2. **The amount is in USDC, the sale is in EUR.** Nothing in the protocol records the EUR price or the rate at sale time; the seller must log both itself, and the realised EUR (I1/I2) will differ from the quoted EUR by the spread and fees. Finance reconciles in EUR, so the seller log — not the receipt — is the book of record.
3. **Settlement time is chain time, not sale time.** With `exact` they are seconds apart; with a held `upto` authorization (Suite D, up to 48 h) the settlement can land days after the dispense, and in a different accounting period. Book the sale on the seller's timestamp and treat the settlement as the cash event.

With those three logged by the seller, reconciliation is a join on payment id; without them it is a manual match of 0.001 USDC lines against a sales list, which does not scale past a test.

Design changes these results force

Engineering consequences, separated from the go/no-go because they bind whoever builds the machine rather than whoever decides to. Each one is a measured result in this report, not a recommendation on principle.

1. **Gate the dispense on settlement, not verify.** On the public facilitator a replayed payment *passes* `/verify` and the handler runs; only settlement rejects it (Suite E3). CDP happens to catch the replay at verify — but a dispense wired to verify is safe only on facilitators that simulate, so gate on settlement.
2. **Use upto, not exact.** A reused `upto` authorization is refused at verify with `permit2_simulation_failed` — nothing broadcast, no gas. The same reuse under `exact` is caught only after broadcast: mined, reverted, 40,883 gas burned from the facilitator's wallet. `upto` also supports partial settlement and a free release.
3. **Reconcile on amount and transaction, never success alone.** A zero settlement returns `success: true` with an empty transaction, so "released" and "charged" are indistinguishable from the flag (Suite D2).
4. **Do not override the SDK's Cache-Control: private** on paid responses. `@x402/hono` sets it via `withPrivateCacheControl`; a Cloudflare Cache Rule with an Edge Cache TTL override ignores origin cache headers and would serve paid content to an unpaid client.

5. **Budget 3 subrequests per paid request, not 2.** Verify and settle are one each, and a cold isolate adds a `/supported` call, so a tight `limits.subrequests` fails intermittently in step with isolate churn (B5).
6. **Do not read `invalid_exact_evm_signature` at settle as a client bug.** See the settlement-failure section: the label accuses the client, and the client is what the records exonerate.
7. **An MCP surface is separate work.** x402 over MCP carries payment in JSON-RPC `_meta` rather than HTTP headers, so an MCP-paying agent cannot pay an HTTP route at all — a second endpoint, not a config flag.
8. **The agent clients' spending cap is the entire security control.** `withX402Client` holds a private key and pays on a 402 automatically; its default cap is 0.10 USDC.
9. **Reject any price below \$0.000001 before it reaches the SDK.** `@x402/core` truncates sub-unit prices to zero without an error, and a zero-amount payment settles as `success: true` with content delivered (C3). A computed price that underflows is a free dispense; a startup assertion on the atomic amount costs one line.
10. **Pin `facilitatorAddress` before using `upto` on CDP behind more than one isolate.** CDP advertises a different pool wallet per `/supported` call; a payment signed against one isolate's 402 is refused by another's with `No matching payment requirements` (15 of 18 on the deployed Worker, Suite D). `exact` is unaffected because its `extra` carries no address.

Seller-side summary

Seller records: 27833. Verified: 11928. Settled: 4271. Settle failures: 7657. Canceled after handler error: 0. Settled despite cancel: 0.

Every figure is reported per host, per facilitator and per network; blended numbers are never used. Records produced by harness defects or funding artifacts are excluded and documented in `results/README.md` and `results/quarantine/`.